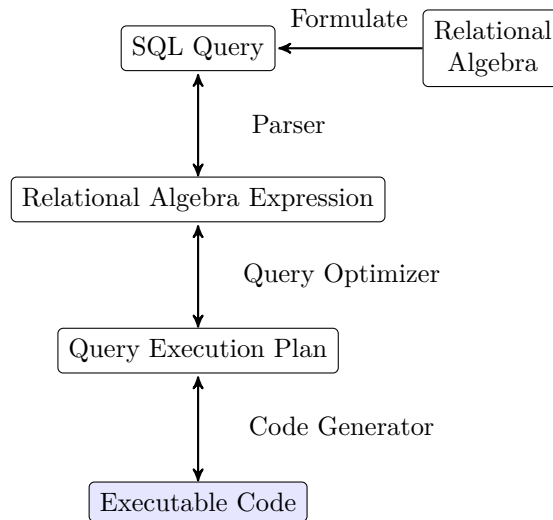


Query Processing

Weihai Yu

October 10, 2019

Query processing



Measures of cost

$$S \cdot t_S + b \cdot t_T$$

S number of seeks

b number of block transfers

t_S seek time (actually access time), $\sim 10\text{ms}$

t_T block transfer time, $\sim 0.1\text{ms}$

a write costs more than a read (data are read back for validation), but we ignore this distinction

Selection

- linear search
 - $t_S + b_r \cdot t_T$
- primary B⁺-tree index
 - $h_i \cdot (t_S + t_T) + t_S + b \cdot t_T$
- hash-organized file
 - $(t_S + b_H \cdot t_T) \cdot (1 + n_{overflow})$
- secondary B⁺-tree index
 - $h_i \cdot (t_S + t_T) + b_i + n \cdot (t_S + t_T)$
- hash-index
 - $(t_S + b_h \cdot t_T) \cdot (1 + n_{overflow}) + n \cdot (t_S + t_T)$

b_H : number of buckets in hash-organized file. b_h : number of buckets in hash-index file. b_H is greater than b because the blocks in the bucket include records that have the same hash value but different search-key values.

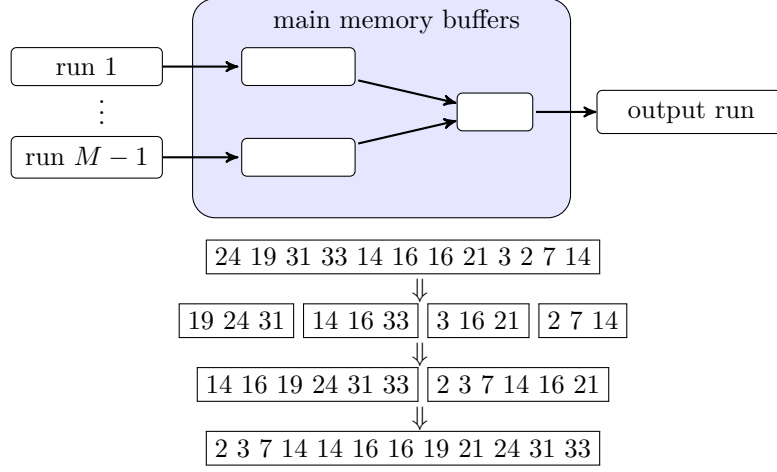
Complex selection

- conjunction $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
 - use one index
 - use a multi-attribute index
 - use several indices and intersection of pointers
- disjunction $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$
 - if all have index, union of pointers
 - linear search (most of the time)
- negation $\sigma_{\neg \theta}(r)$
 - linear search

Sorting

- order by, group by
- distinct
- ...
- what if the relation is larger than the main memory?

External sort-merge



Sort-merge cost

M buffer pages

B blocks ($B > M$)

- initially $\lceil \frac{B}{M} \rceil$ runs
 - $2B$ block transfers, $2\lceil \frac{B}{M} \rceil$ seeks
- number of merge passes: $\lceil \log_{M-1}(\frac{B}{M}) \rceil$
 - each pass: $2B$ block transfers, $2B$ seeks
- $2B(\lceil \log_{M-1}(\frac{B}{M}) \rceil + 1)$ block transfers
- $2B\lceil \log_{M-1}(\frac{B}{M}) \rceil + 2\lceil \frac{B}{M} \rceil$ seeks

Nested-loop join $r \bowtie_{\theta} s$

```

foreach tuple  $t_r$  in  $r$  do
  foreach tuple  $t_s$  in  $s$  do
    if  $\theta(t_r, t_s)$  then output  $t_r \cdot t_s$ 
  end
end

```

- r has b_r blocks, n_r tuples
- s has b_s blocks, n_s tuples
- $b_r + n_r \cdot b_s$ block transfers
- $b_r + n_r$ seeks

- plus, we may need to write the result to disk
 - this is the same for all join algorithms

Block nested-loop join $r \bowtie_{\theta} s$

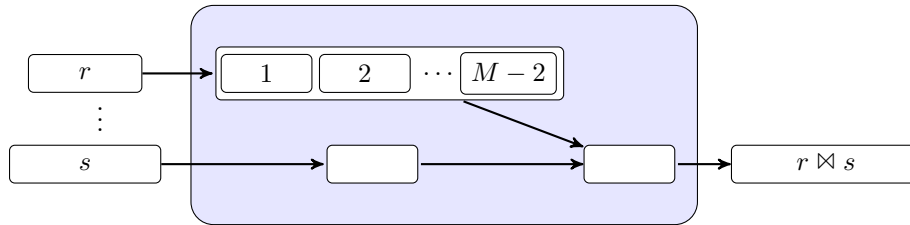
```

foreach block  $B_r$  in  $r$  do
  foreach block  $B_s$  in  $s$  do
    foreach tuple  $t_r$  in  $B_r$  do
      foreach tuple  $t_s$  in  $B_s$  do
        if  $\theta(t_r, t_s)$  then output  $t_r \cdot t_s$ 
      end
    end
  end
end
end

```

- $b_r + b_r \cdot b_s$ block transfers
 - $2b_r$ seeks
 - choose smaller table for outer loop
 - inner loop terminates early if s is sorted
 - or indexed nested-loop join
 - how many pages do we need to perform this algorithm?
- 3

Block nested-loop join $r \bowtie_{\theta} s$, M buffer pages



- $b_r + \lceil \frac{b_r}{M-2} \rceil \cdot b_s$ block transfers
- $2 \lceil \frac{b_r}{M-2} \rceil$ seeks

Indexed nested-loop join

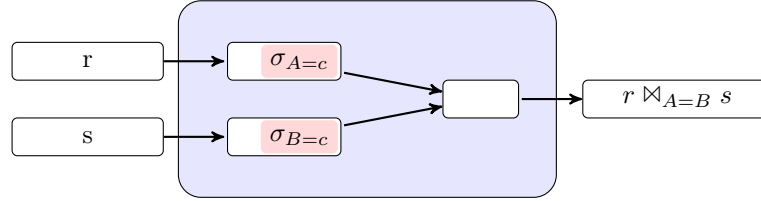
```

foreach tuple  $t_r$  in  $r$  do
  foreach tuple  $t_s$  in  $s$  obtained
    with index for  $\theta(t_r, t_s)$  do
    output  $t_r \cdot t_s$ 
  end
end

```

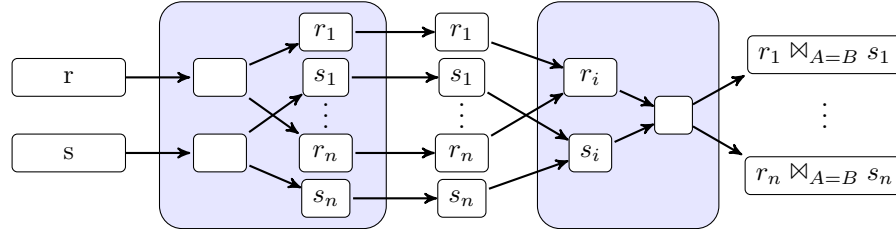
- $b_r(t_S + t_T) + n_r \cdot c$
- secondary B⁺-tree index $c = (h_i + n_\theta) \cdot (t_S + t_T)$
- compare with $2\lceil \frac{b_r}{M-2} \rceil t_S + (b_r + \lceil \frac{b_r}{M-2} \rceil b_s) t_T$
- t_S : $b_r + n_r \cdot (h_i + n_\theta) \gg 2\lceil \frac{b_r}{M-2} \rceil$
- t_T : $n_r \cdot (h_i + n_\theta)$ vs. $\lceil \frac{b_r}{M-2} \rceil b_s$
- not worthwhile unless s is very large and $\theta(t_r, t_s)$ is very selective

Merge join



- r and s are sorted on A and B
- $b_r + b_s$ block transfers
- $\lceil \frac{b_r}{b} \rceil + \lceil \frac{b_s}{b} \rceil$ seeks (what is b ?)
- applicable for equi-joins and natural joins

Hash join $r \bowtie_{A=B} s$



- two stages: partition using a hash function, join the partitions
- applicable to equi- and natural-joins

- good hash function: every probing partition can stay in memory
 - overflow resolution and overflow avoidance
- $3(b_r + b_s) + 4n$ block transfers (each partition may have a partially filled block)
- $2(\lceil \frac{b_r}{b} \rceil + \lceil \frac{b_s}{b} \rceil) + 2n$ seeks

Duplicate elimination

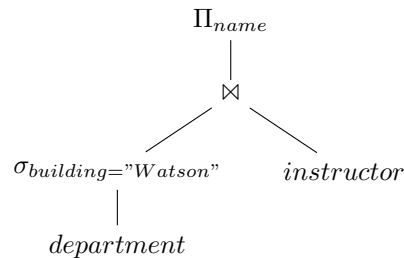
- sort: duplicates can be deleted during run generation and merge steps
- hash partition and eliminate in each partition
- projection may need duplicate elimination

Set operations

- $\cup, \cap, -$
- sort and “merge”
- hash and “merge”

Expression with multiple operations

$\Pi_{name}(\sigma_{building="Watson"}(department) \bowtie instructor)$



- materialization
 - start from the lowest level operations
 - store results in temporary relations
 - process higher-level operations, until the top-level operation is done
 - writing and reading temporary results are expensive
 - *double buffering*: perform CPU work while doing disk IOs
- pipelining
 - pushing, eager, producer-driven
 - pulling, lazy, demand-driven